Technical Design Document

CampusGPT

An AI-Powered University Intelligence and Student Assistance Platform

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Specialization: **Cyber Security**

Prepared by

Name	SAP ID	Batch
Ankur Karn	500118986	11
Ashmita Sharma	500122422	11
Ananya Singh	500123837	11
Ayushi Mishra	500121986	11

Under the guidance of

Mr. Yogesh Rathia

Project Mentor

School of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES

Dehradun – 248007, Uttarakhand

October 2026 | Version 1.0

Contents

Revision History **v**

Document Conventions **vi**

1 Introduction **1**

1.1 Purpose 1

1.2 Evaluation Components 1

1.3 Design Principles 1

2 Technical Diagrams **3**

2.1 System Architecture 3

2.2 Data Flow Diagram (Level 1) 4

2.3 Sequence Diagram: Answering a Question 4

2.4 Database Design (ER Diagrams) 6

3 Inter-Process Communication **8**

3.1 Processes in the System 8

3.2 IPC Mechanisms 8

3.3 Worked Example: Document Ingestion 10

3.4 Streaming Answers to the Browser 11

3.5 Synchronisation and Failure Handling 11

4 Programming Concepts **12**

5 Libraries and Frameworks **15**

References **17**

List of Figures

2.1 System architecture of CampusGPT. 3
2.2 Level-1 Data Flow Diagram of CampusGPT. 4
2.3 Sequence diagram for a question that needs both records and policy. 5
2.4 ER diagram: academic records. 6
2.5 ER diagram: authentication, documents and chat. 7

3.1 Process view with inter-process communication channels. 8
3.2 State machine of an ingestion job. 10

List of Tables

1.1 Evaluation components and where they are addressed 1

2.1 Planned table sizes for 5,000 synthetic students (approximate) 7

3.1 IPC mechanisms used in CampusGPT 9

4.1 Programming concepts and where they are applied 12

5.1 Libraries, their purpose, and the reason for each choice 15

List of Listings

3.1 Enqueueing and claiming a job (shared table and NOTIFY) 10
3.2 Worker loop: LISTEN with a polling safety net 11
4.1 Abstraction and polymorphism: the LLM provider adapter 13
4.2 Decorator-based tool registry with server-side permission checks 14
4.3 Dependency injection and an async generator streaming SSE events 14
4.4 Deterministic attendance calculation with exact arithmetic 14

Revision History

Date	Change	Reason for Changes
Tue, 13 Oct 2026	Version 1.0. Technical diagrams, inter-process communication design, programming concepts and library selection.	First submission for evaluation, alongside SRS version 1.1.

Document Conventions

- This document describes *how* CampusGPT is built. *What* it must do is specified in the Software Requirements Specification (SRS); requirement IDs such as **FR-08** refer to that document.
- Code listings are design sketches in Python and SQL. They show the intended structure; identifiers may change during implementation.
- Numbered channel markers such as **(3)** identify the same inter-process communication channel in every diagram and table.
- All student records and policy documents are synthetic (fictional).

CHAPTER 1

Introduction

1.1 Purpose

CampusGPT is an AI-powered helpdesk that answers university students' questions from two sources: their own records, read through permission-checked backend tools, and policy documents, searched with Retrieval-Augmented Generation (RAG) [1]. This document presents the technical design: the system's structure and data, the processes that make up the running system and how they communicate, the programming concepts used to build it, and the libraries chosen.

1.2 Evaluation Components

Table 1.1 shows where each evaluated component is addressed.

Table 1.1: Evaluation components and where they are addressed

Component	Content	Chapter
Technical Diagram	System architecture, Level-1 DFD, sequence diagram, ER diagrams, process view, job state machine	2, 3
Programming Concepts	Object-oriented design, design patterns, asynchronous programming, generators, validation, transactions, data structures, testing	4
Inter-Process Communication	Seven IPC mechanisms across five processes, with the document-ingestion pipeline as the worked example	3
Libraries	Each library's purpose, the alternative considered, and the reason for the choice	5
SRS Document	Submitted separately (version 1.1)	–

1.3 Design Principles

1. **The language model never touches the database.** It can only request one of a fixed set of read-only tools; the backend checks permission and supplies the student's identity from the login session.
2. **Facts and calculations come from code.** Numbers are read from PostgreSQL and computed deterministically; the model selects tools and words the answer.
3. **Slow work runs in a separate process.** Document ingestion is handed to a worker process so that chat requests are never blocked.

4. **One database for everything.** PostgreSQL stores records, vectors, full-text indexes and the job queue, and also carries notifications between processes.
5. **Replaceable external parts.** The LLM provider and the university data source sit behind interfaces and can be swapped by configuration.

CHAPTER 2

Technical Diagrams

2.1 System Architecture

Figure 2.1 shows CampusGPT as four tiers. Grey arrows are function calls inside one process; black arrows, numbered **(1)**–**(4)**, cross a process boundary and are detailed in Chapter 3.

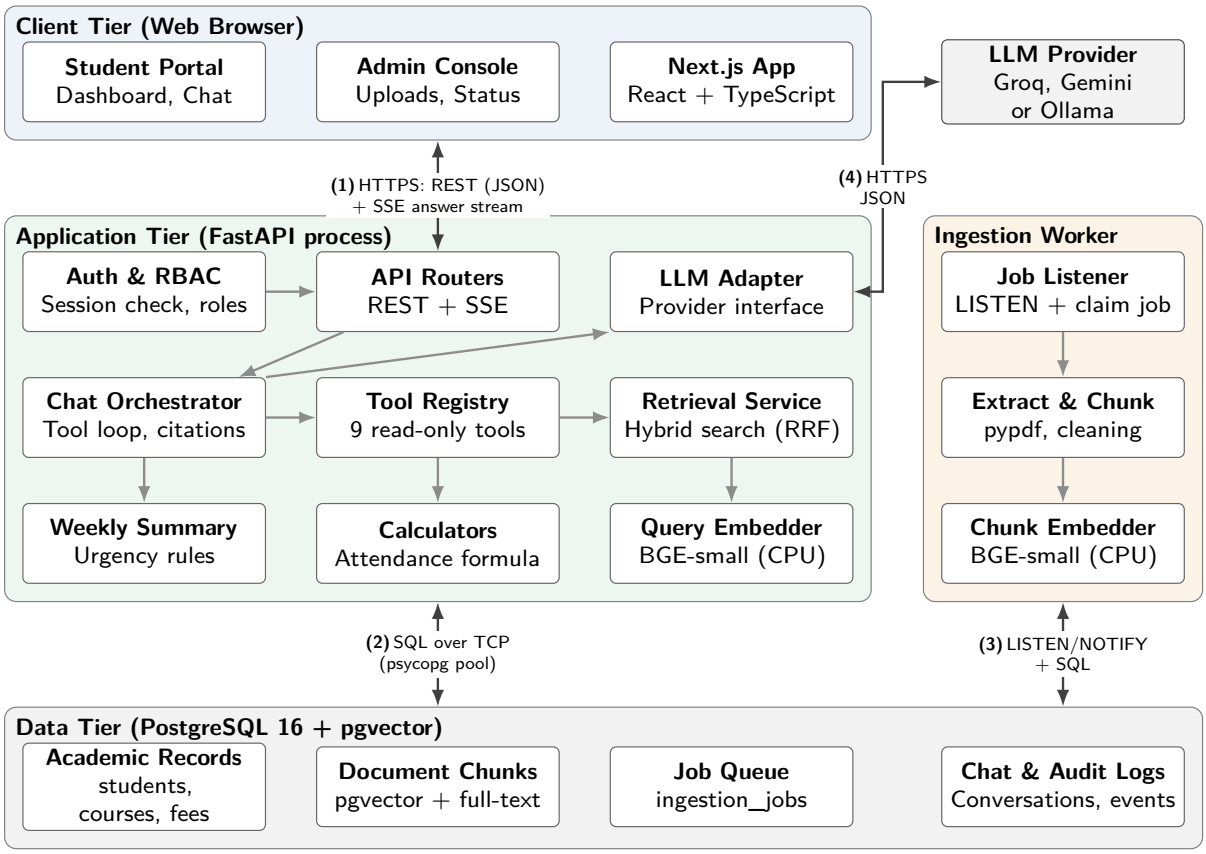


Figure 2.1: System architecture of CampusGPT.

- **Client tier.** A Next.js application with a student portal (dashboard and chat) and an admin console (document upload and status).
- **Application tier.** One FastAPI process. Every request passes the authentication and role check first. The chat orchestrator runs the tool loop with the LLM, the tool registry executes permitted tools, and the retrieval service performs hybrid search using a locally loaded embedding model.
- **Ingestion worker.** A separate Python process that converts uploaded documents into searchable chunks, so CPU-heavy embedding never slows down chat.
- **Data tier.** PostgreSQL 16 with the pgvector extension [2] holds academic records, document chunks with vectors, the job queue and logs.

2.2 Data Flow Diagram (Level 1)

The Level-0 context diagram appears in the SRS. Figure 2.2 decomposes the system into five processes and five data stores, following DeMarco's structured-analysis notation [3]. Every data flow passes through a process; no external entity reads or writes a data store directly.

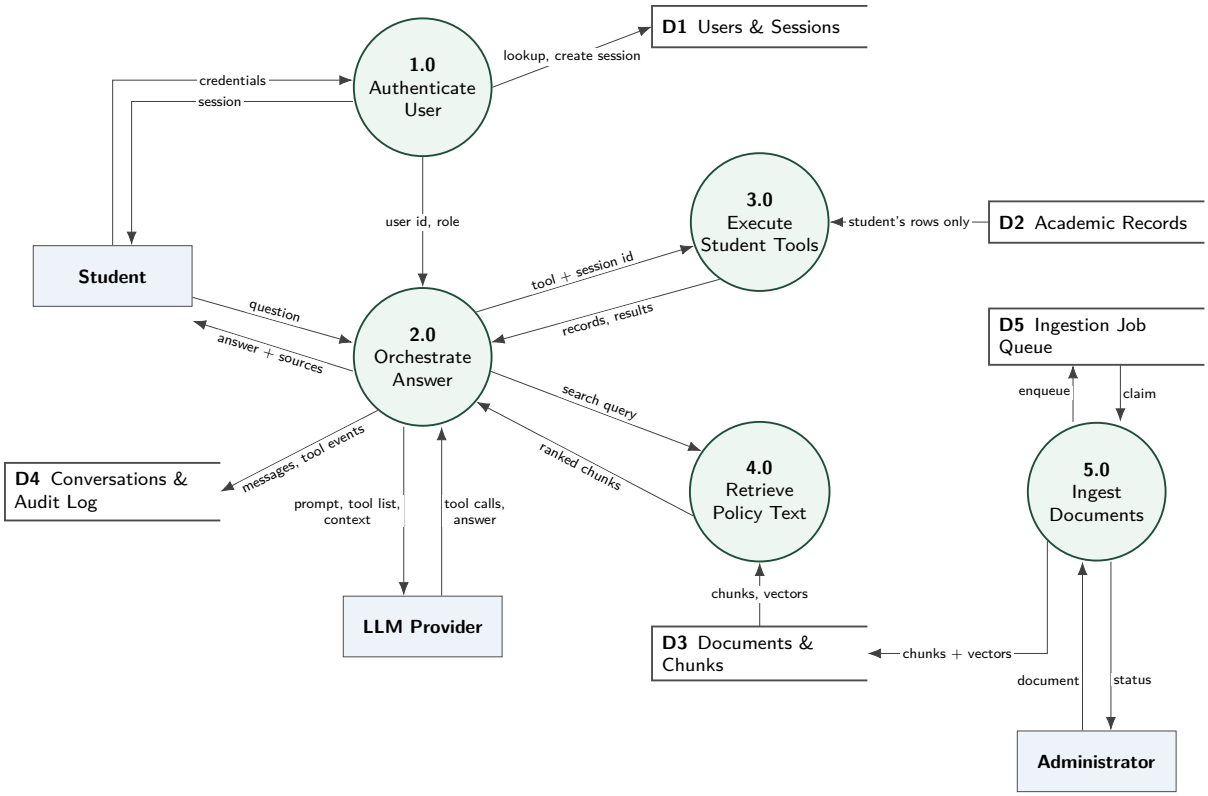


Figure 2.2: Level-1 Data Flow Diagram of CampusGPT.

2.3 Sequence Diagram: Answering a Question

Figure 2.3 traces the question "Can I sit for my DBMS exam?" from a student with 41 of 60 classes attended. The orchestrator first obtains the student's attendance, then the relevant policy text, and only then asks the model for an answer. The student identifier is added by the orchestrator, never by the model, and every citation is checked against the chunks that were actually retrieved.

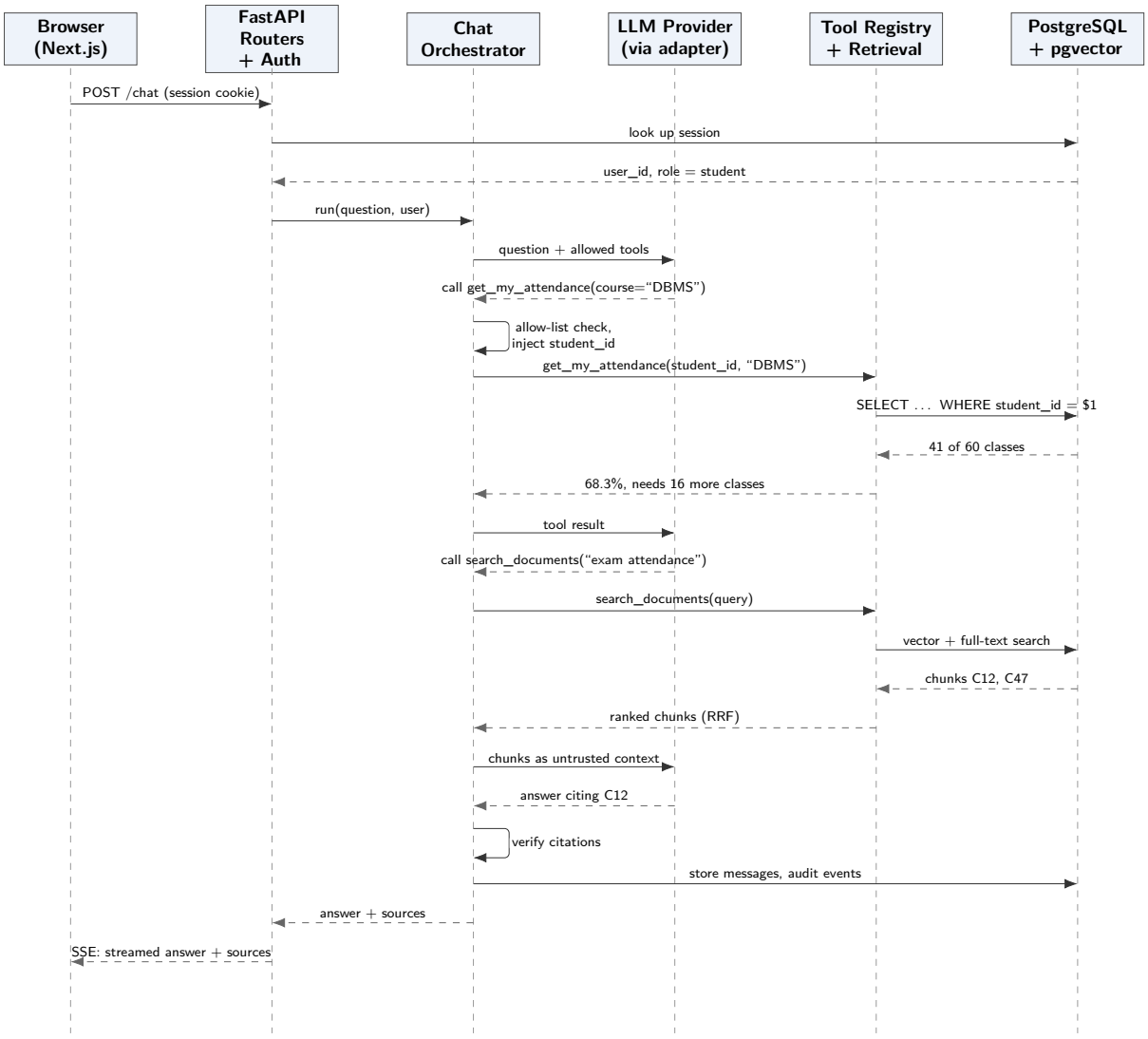


Figure 2.3: Sequence diagram for a question that needs both records and policy.

2.4 Database Design (ER Diagrams)

The schema follows the relational model [4] and is drawn in entity–relationship form [5]. Figure 2.4 shows the academic records and Figure 2.5 the tables supporting authentication, documents and chat. The `users` table appears in both figures.

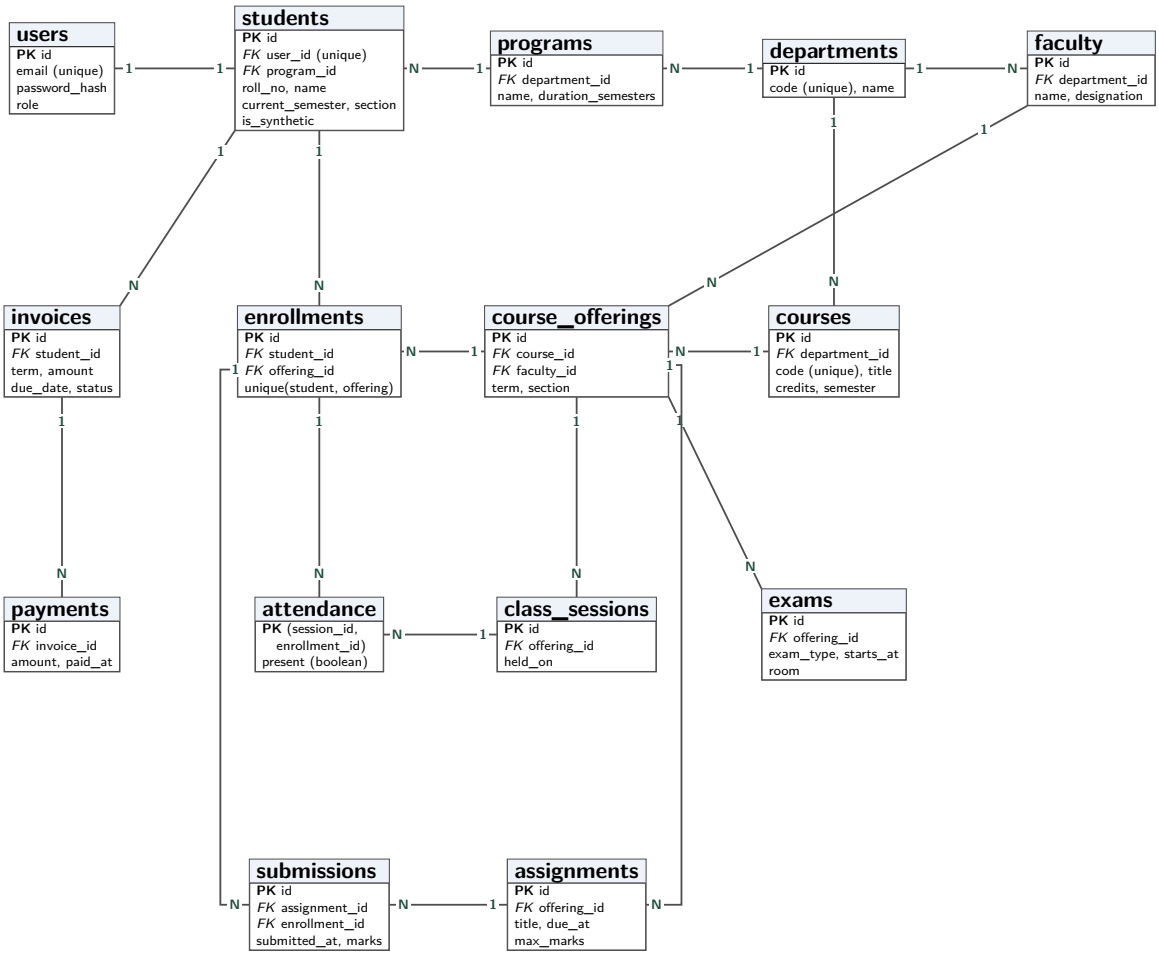


Figure 2.4: ER diagram: academic records.

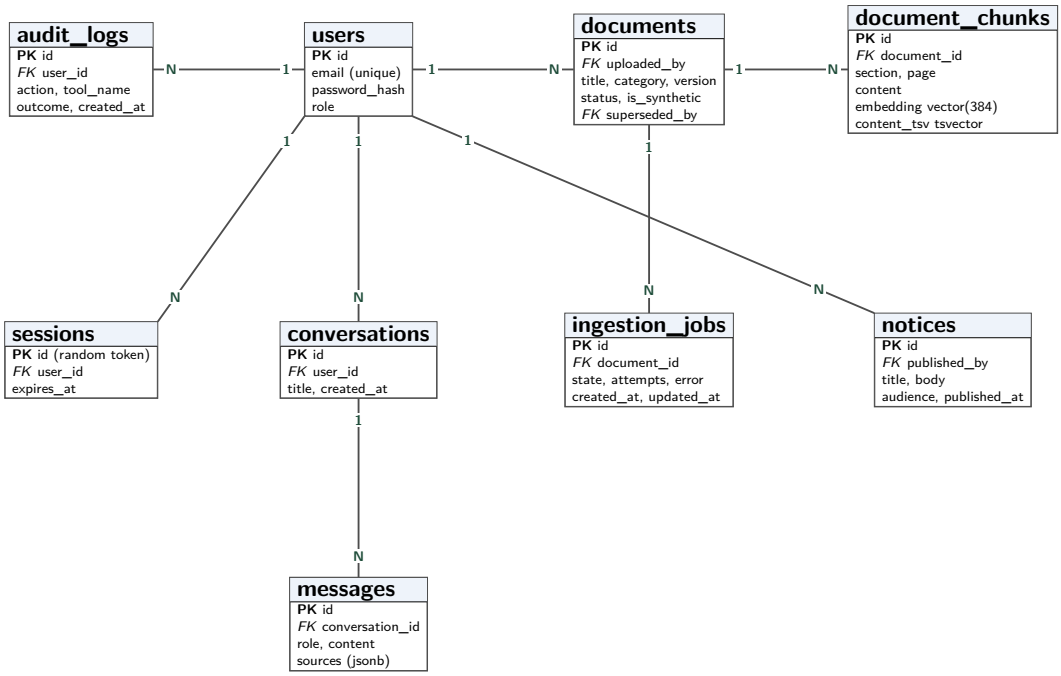


Figure 2.5: ER diagram: authentication, documents and chat.

Design decisions.

- The schema is in third normal form. Derived values such as attendance percentage are computed by a view (`attendance_summary`) instead of being stored, so they can never disagree with the underlying records.
- Attendance is stored per class session, which allows the calculator to answer "how many more classes" exactly and lets the generator create realistic patterns.
- Integrity is enforced by the database: foreign keys, `UNIQUE` constraints (one enrollment per student per offering), and `CHECK` constraints (amounts ≥ 0, marks $\leq$ maximum).
- Indexes: B-tree indexes on every foreign key and on `(student_id)` filters; an HNSW index [6] on `embedding`; a GIN index on `content_tsv` for full-text search.
- Every generated row carries `is_synthetic = true`, as required by FR-02.

Table 2.1 lists the planned dataset size for 5,000 students.

Table 2.1: Planned table sizes for 5,000 synthetic students (approximate)

Table	**Rows**	**Table**	**Rows**
students	5,000	class_sessions	18,000
faculty	150	attendance	1,350,000
courses	180	assignments	1,600
course_offerings	400	submissions	110,000
enrollments	30,000	exams	800
invoices	10,000	payments	9,000

CHAPTER 3

Inter-Process Communication

3.1 Processes in the System

At run time CampusGPT consists of five cooperating operating-system processes on the server side of the browser (Figure 3.1): the Next.js server, the FastAPI server, the ingestion worker, the PostgreSQL server (which itself forks one backend process per connection), and the remote LLM service. They share no memory; all cooperation happens through the channels numbered **(1)**–**(7)**.

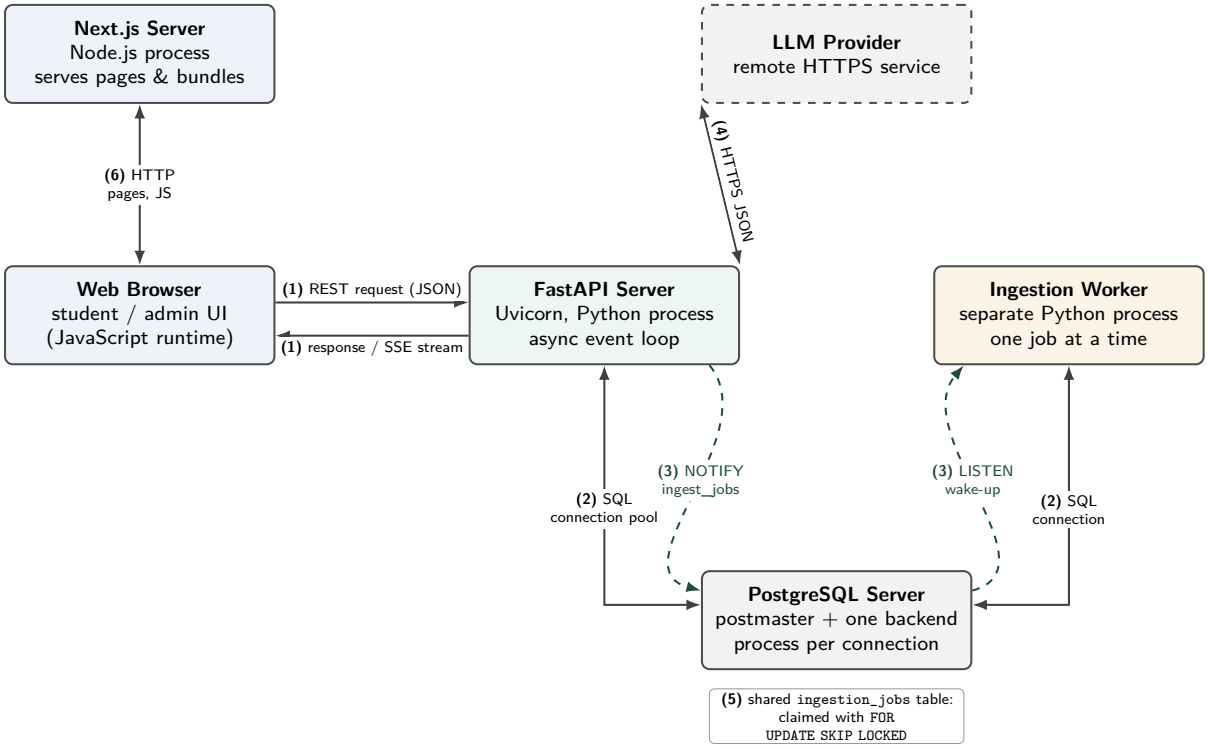


Figure 3.1: Process view with inter-process communication channels.

3.2 IPC Mechanisms

Table 3.1 lists every mechanism, the processes it connects, its communication pattern, and the reason for choosing it [7].

Table 3.1: IPC mechanisms used in CampusGPT

#	Mechanism	Between	Pattern	Why
(1)	HTTP REST with JSON [8], plus Server-Sent Events [9]	Browser ↔ FastAPI	Request–response; server push stream	Stateless and universal. SSE streams the answer as it is generated, which lowers perceived latency without the complexity of WebSockets.
(2)	PostgreSQL wire protocol over TCP or a Unix domain socket	FastAPI, worker ↔ PostgreSQL	Request–response through a connection pool	Reusing connections avoids a new server process per query; parameterised queries prevent SQL injection.
(3)	`LISTEN/NOTIFY` [10] on channel `ingest_jobs`	FastAPI → PostgreSQL → worker	Asynchronous publish–subscribe	Wakes the worker instantly without busy polling. A notification sent inside a transaction is delivered only if the transaction commits.
(4)	HTTPS with JSON	FastAPI → LLM provider	Remote request–response with timeout and retry	The only channel that leaves the machine; carries the question, tool definitions and retrieved text, never raw database access.
(5)	Shared `ingestion_jobs` table with row locks (`FOR UPDATE SKIP LOCKED`) [11]	FastAPI, worker	Shared state with mutual exclusion	A durable queue: jobs survive crashes, and each job is claimed by exactly one worker even if several run.
(6)	HTTP	Browser ↔ Next.js server	Request–response	Delivers pages and JavaScript bundles.
(7)	POSIX signals (`SIGTERM`, `SIGINT`)	Operating system / process manager → FastAPI, worker	Asynchronous notification	Graceful shutdown: the worker finishes the current job and the server drains open requests before exiting.

Why NOTIFY together with a table. PostgreSQL does not store notifications for processes that are not listening, so `NOTIFY` alone would lose jobs uploaded while the worker is stopped. The table is therefore the durable source of truth, and the notification is only a wake-up signal. As a safety net, the worker also checks the table every 30 seconds.

Why not a separate message broker. Redis, RabbitMQ or Celery would add another server process and its memory to an 8 GB development laptop, while PostgreSQL already provides durable storage, row locking and publish–subscribe.

3.3 Worked Example: Document Ingestion

Figure 3.2 shows the life cycle of an ingestion job, and the steps below follow one upload across process boundaries.

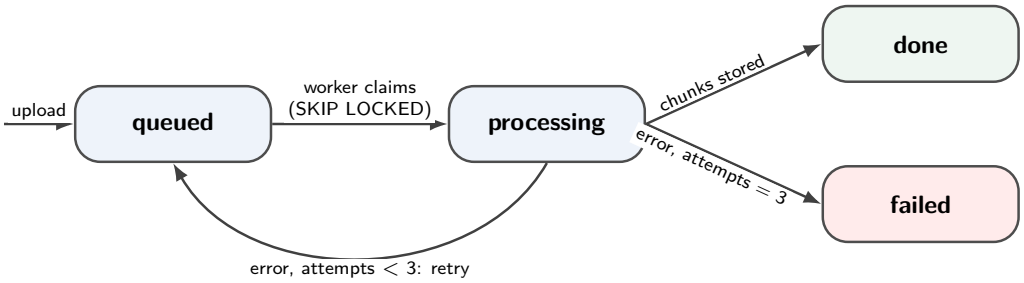


Figure 3.2: State machine of an ingestion job.

Ingestion across processes

1. **Browser → FastAPI (1):** the administrator uploads a PDF with `POST /admin/documents`.
2. **FastAPI → PostgreSQL (2):** in one transaction, insert the `documents` row and a `queued` job, then execute `NOTIFY ingest_jobs` **(3)**. The API replies `202 Accepted` immediately.
3. **PostgreSQL → worker (3):** on commit, the listening worker wakes up.
4. **Worker → PostgreSQL (5):** claim the oldest queued job with `FOR UPDATE SKIP LOCKED` and set it to `processing`.
5. **Worker (local):** extract text, clean it, split it into chunks and embed them.
6. **Worker → PostgreSQL (2):** in one transaction, insert all chunks, mark the previous document version as superseded, and set the job to `done`.
7. **Browser → FastAPI (1):** the admin console polls `GET /admin/documents/{id}` every two seconds and shows the new status.

Listing 3.1: Enqueueing and claiming a job (shared table and NOTIFY)

```
-- FastAPI, inside the upload transaction
INSERT INTO ingestion_jobs (document_id, state) VALUES (:doc_id, 'queued');
SELECT pg_notify('ingest_jobs', :doc_id::text);   -- delivered on COMMIT

-- Worker: claim one job; rows locked by another worker are skipped
UPDATE ingestion_jobs
   SET state = 'processing', attempts = attempts + 1, updated_at = now()
 WHERE id = (SELECT id FROM ingestion_jobs
             WHERE state = 'queued'
             ORDER BY created_at
             LIMIT 1
             FOR UPDATE SKIP LOCKED)
RETURNING id, document_id;
```

Listing 3.2: Worker loop: LISTEN with a polling safety net

```
async def run_worker(dsn: str) -> None:
    conn = await psycopg.AsyncConnection.connect(dsn, autocommit=True)
    await conn.execute("LISTEN ingest_jobs")
    while not shutdown.is_set():                    # set by the SIGTERM handler
        while (job := await claim_job(conn)) is not None:
            await process(job)                      # extract, chunk, embed, store
        # Sleep until a notification arrives, or 30 s at most
        async for _ in conn.notifies(timeout=30, stop_after=1):
            pass
```

3.4 Streaming Answers to the Browser

For each question, FastAPI returns a `text/event-stream` response. The orchestrator emits typed events: `status` (for example "Checking your attendance…"), `token` (a piece of the answer), `sources` (database and document citations), `done` and `error`. Because the browser's `EventSource` API supports only `GET`, the frontend sends a `POST` with `fetch` and reads the event stream from the response body.

3.5 Synchronisation and Failure Handling

- **Timeouts and retries.** LLM calls time out after 30 seconds and are retried at most twice with exponential back-off; afterwards the chat shows a clear error while all other pages keep working (FR-16).
- **Worker crash.** A job left in `processing` for more than ten minutes is returned to `queued` on the next claim, up to three attempts.
- **Atomicity.** Chunks and the job status are written in one transaction, so a failure never leaves a half-indexed document visible to search.
- **Back-pressure.** Connection pools cap the number of PostgreSQL backend processes; rate limits cap chat requests per user (NFR-S5).

CHAPTER 4

Programming Concepts

Table 4.1 lists the programming concepts used in CampusGPT and where each appears; selected concepts are illustrated by the listings that follow.

Table 4.1: Programming concepts and where they are applied

Concept	**Where it is used**	**Purpose**
Abstraction, inheritance, polymorphism	`LLMProvider` abstract class with Groq, Gemini and Ollama subclasses (Listing 4.1)	Orchestrator code is identical for every provider.
Design patterns [12]	Adapter (LLM provider, university data source), Strategy (provider chosen by configuration), Registry (tools), Repository (data access), Publish–subscribe (`LISTEN/NOTIFY`)	Known solutions with clear responsibilities.
Decorators and higher-order functions [13]	`@tool` registration (Listing 4.2); FastAPI route decorators	Declarative registration without boilerplate.
Dependency injection	FastAPI `Depends` for the current user and database session	Testable handlers; authentication cannot be forgotten.
Asynchronous programming [14]	`async/await` handlers, database driver and HTTP client	Many concurrent requests on one process while waiting on I/O.
Generators and streaming [15]	Async generator producing SSE events (Listing 4.3); batch generators in the data generator	Constant memory, output sent as soon as it exists.
Type hints and validation	Pydantic models for requests, responses and tool arguments; TypeScript on the frontend	Invalid input rejected at the boundary; JSON Schema for tool definitions generated automatically.
Exception handling	Custom hierarchy (`ToolNotPermitted`, `ToolValidationError`, `ProviderUnavailable`) mapped to HTTP status codes	Consistent, safe error messages.
Transactions and ACID	Upload, ingestion and chat logging run as single transactions	No partial writes.
Concurrency control	Row locks with `SKIP LOCKED`; `UNIQUE` constraints for idempotent inserts	Correct behaviour with several workers.

Concept	Where it is used	Purpose
Data structures and algorithms	Hash map (tool registry), binary heap (weekly-summary priority queue), Reciprocal Rank Fusion [16], cosine similarity, sliding-window chunking	Efficient lookup, ranking and retrieval.
Exact arithmetic	`fractions.Fraction` in the attendance calculator (Listing 4.4)	No floating-point rounding errors in eligibility decisions.
Secure programming	Argon2id hashing [17], CSPRNG session tokens (`secrets`), parameterised SQL, role checks [18]	Defence against credential theft, injection and privilege escalation.
Text processing	Regular expressions for cleaning extracted PDF text	Removes headers, footers and broken hyphenation before chunking.
Reproducible randomness	Seeded Faker and NumPy generators	Identical synthetic dataset on every machine.
Automated testing	pytest fixtures, parametrised tests, a rolled-back transaction per test	Fast, isolated, repeatable tests.
Frontend components and hooks	React components, `useState`/`useEffect`, stream reading with `ReadableStream`	Responsive UI that renders answers while they stream.

Listing 4.1: Abstraction and polymorphism: the LLM provider adapter

```
class LLMProvider(ABC):
    """Interface every model provider implements."""

    @abstractmethod
    async def complete(self, messages: list[Message],
                       tools: list[ToolSpec]) -> LLMReply: ...

class GroqProvider(LLMProvider):
    async def complete(self, messages, tools):
        resp = await self.http.post("/chat/completions",
                                    json=to_openai_format(messages, tools))
        resp.raise_for_status()
        return LLMReply.from_openai_format(resp.json())

def get_provider(settings: Settings) -> LLMProvider:      # Strategy by configuration
    providers = {"groq": GroqProvider, "gemini": GeminiProvider,
                 "ollama": OllamaProvider}
    return providers[settings.llm_provider](settings)
```

Listing 4.2: Decorator-based tool registry with server-side permission checks

```
TOOLS: dict[str, Tool] = {}

def tool(name: str, roles: set[str], args: type[BaseModel]):
    def register(fn):
        TOOLS[name] = Tool(name=name, roles=roles, args=args, fn=fn)
        return fn
    return register

class AttendanceArgs(BaseModel):
    course: str | None = None

@tool("get_my_attendance", roles={"student"}, args=AttendanceArgs)
async def get_my_attendance(db: AsyncSession, student_id: int, a: AttendanceArgs):
    return await attendance_repo.summary(db, student_id, course=a.course)

async def execute(call: ToolCall, user: User, db: AsyncSession):
    spec = TOOLS.get(call.name)
    if spec is None or user.role not in spec.roles:
        raise ToolNotPermitted(call.name)            # the model cannot bypass this
    args = spec.args.model_validate(call.arguments)  # schema validation
    return await spec.fn(db, user.student_id, args)  # identity from the session
```

Listing 4.3: Dependency injection and an async generator streaming SSE events

```
@router.post("/chat")
async def chat(req: ChatRequest,
               user: User = Depends(current_user),     # 401 if not logged in
               db: AsyncSession = Depends(get_db)):
    async def events():
        async for event in orchestrator.run(req.question, user, db):
            yield {"event": event.kind, "data": event.model_dump_json()}
    return EventSourceResponse(events())
```

Listing 4.4: Deterministic attendance calculation with exact arithmetic

```
def classes_needed(attended: int, held: int,
                   target: Fraction = Fraction(3, 4)) -> int:
    """Smallest x such that (attended + x) / (held + x) >= target."""
    if held == 0 or Fraction(attended, held) >= target:
        return 0
    return math.ceil((target * held - attended) / (1 - target))

assert classes_needed(41, 60) == 16      # 57 / 76 = 75%
```

CHAPTER 5

Libraries and Frameworks

Table 5.1 lists every library CampusGPT depends on. All are free and open source; exact versions are pinned in `requirements.txt` and `package-lock.json`.

Table 5.1: Libraries, their purpose, and the reason for each choice

Library	**Purpose in CampusGPT**	**Alternative**	**Reason for choice**
Backend			
FastAPI [19]	HTTP API, routing, dependency injection, OpenAPI documentation	Flask, Django	Asynchronous by design; validation and API docs built in.
Uvicorn	ASGI server that runs the FastAPI process	Gunicorn (WSGI)	Standard asynchronous server for FastAPI.
Pydantic [20]	Schemas for requests, responses and tool arguments	marshmallow	Integrated with FastAPI; exports JSON Schema used as LLM tool definitions.
SQLAlchemy 2 [21]	ORM and query builder	Raw SQL strings	Typed, composable queries with bound parameters.
psycopg 3 [22]	PostgreSQL driver: async queries, `LISTEN/NOTIFY`, `COPY`	asyncpg	Supports all three IPC features we need and is SQLAlchemy's standard dialect.
Alembic	Versioned database migrations	Hand-written SQL scripts	Repeatable, reversible schema changes.
argon2-cffi	Password hashing with Argon2id	bcrypt	Memory-hard algorithm recommended by OWASP [23].
httpx	Async HTTP client for LLM providers	requests, vendor SDKs	Non-blocking; one client for all providers avoids vendor lock-in.
sse-starlette	Server-Sent Events responses	WebSockets	One-way streaming is enough and simpler to secure.
slowapi	Rate limiting per user and IP	Custom middleware	Decorator-based limits per route.
AI and document processing			
sentence-transformers [24]	Runs the BGE-small embedding model [25] on the CPU	Paid embedding APIs	Free, offline, and no document text leaves the machine.

Library	**Purpose in CampusGPT**	**Alternative**	**Reason for choice**
pypdf	PDF text extraction	PyMuPDF, pdfplumber	Pure Python with a permissive licence (PyMuPDF is AGPL).
Synthetic data generation			
Faker (`en_IN`)	Indian names, phone numbers and addresses	Hand-written name lists	Locale-aware, seedable, large variety.
NumPy	Statistical distributions for attendance, marks and fee payments	Python `random`	Fast vectorised sampling with a seeded generator.
Testing and quality			
pytest	Unit, integration and security tests	unittest	Fixtures and parametrisation.
Ruff	Linting and formatting	flake8 + black	One fast tool for both.
Frontend			
Next.js [26]	React framework: routing and production builds	Vite + React	File-based routing and built-in optimisation.
React [27]	Component-based user interface	Vue	Largest ecosystem; required by Next.js.
TypeScript	Static typing for frontend code	JavaScript	Catches type errors at build time.
Tailwind CSS	Styling with a consistent design scale	Bootstrap	Small output, consistent spacing and colour tokens.
react-markdown	Renders formatted chat answers	Raw HTML insertion	Does not render raw HTML, preventing cross-site scripting.
Recharts	Attendance and fee charts on the dashboard	Chart.js	Native React components.
Infrastructure			
PostgreSQL 16 [28]	Relational database, full-text search, job queue, notifications	MySQL, MongoDB	One system provides integrity, search and IPC.
pgvector [2]	Vector column type and HNSW index	Pinecone, ChromaDB, FAISS	No separate vector database; vectors share transactions with their metadata.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS 33)*, 2020, pp. 9459–9474.

[2] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2024. [Online]. Available: `https://github.com/pgvector/pgvector`

[3] T. DeMarco, *Structured Analysis and System Specification*. New York, NY, USA: Yourdon Press, 1978.

[4] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[5] P. P.-S. Chen, "The entity-relationship model—toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[7] W. R. Stevens, *UNIX Network Programming, Volume 2: Interprocess Communications*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[9] WHATWG, "HTML Living Standard: Server-sent events." [Online]. Available: `https://html.spec.whatwg.org/multipage/server-sent-events.html`

[10] The PostgreSQL Global Development Group, "NOTIFY," *PostgreSQL 16 Documentation*. [Online]. Available: `https://www.postgresql.org/docs/16/sql-notify.html`

[11] The PostgreSQL Global Development Group, "SELECT: The locking clause," *PostgreSQL 16 Documentation*. [Online]. Available: `https://www.postgresql.org/docs/16/sql-select.html`

[12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[13] K. D. Smith *et al.*, "PEP 318 – Decorators for functions and methods," 2003. [Online]. Available: `https://peps.python.org/pep-0318/`

[14] Y. Selivanov, "PEP 492 – Coroutines with async and await syntax," 2015. [Online]. Available: `https://peps.python.org/pep-0492/`

[15] Y. Selivanov, "PEP 525 – Asynchronous generators," 2016. [Online]. Available: `https://peps.python.org/pep-0525/`

[16] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR*, 2009, pp. 758–759.

[17] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.

[18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[19] S. Ramírez, "FastAPI documentation." [Online]. Available: `https://fastapi.tiangolo.com/`

[20] Pydantic, "Pydantic documentation." [Online]. Available: `https://docs.pydantic.dev/`

[21] SQLAlchemy, "SQLAlchemy 2.0 documentation." [Online]. Available: `https://docs.sqlalchemy.org/en/20/`

[22] D. Varrazzo, "Psycopg 3 documentation." [Online]. Available: `https://www.psycopg.org/psycopg3/docs/`

[23] OWASP Foundation, "Password Storage Cheat Sheet." [Online]. Available: `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[25] S. Xiao *et al.*, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. 47th ACM SIGIR*, 2024, pp. 641–649.

[26] Vercel, "Next.js documentation." [Online]. Available: `https://nextjs.org/docs`

[27] Meta Open Source, "React documentation." [Online]. Available: `https://react.dev/`

[28] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation." [Online]. Available: `https://www.postgresql.org/docs/16/`